Módulo 03: Deploy a Containerized Backend API to Azure Container Apps

Despliegue Serverless de APIs en Contenedor con Azure Container Apps, Managed Identity y Secretos Runtime

Alineación Curricular Oficial Microsoft Learn: Curso AI-200T00-A: Desarrollo de soluciones en la nube de IA en Azure (5 días / 120 hrs) • **Certificación:** Microsoft Certified: Azure AI Cloud Developer Associate (Examen AI-200) • **Dominio 1:** Desarrollo de soluciones en contenedores en Azure (20% – 25%) [Guía de Estudio Oficial AI-200]

PRERREQUISITOS UNIVERSALES DEL ENTORNO

- **Azure CLI 2.50+ (64-bit):** Requiere arquitectura x64 instalada en `C:\Program Files\Microsoft SDKs\Azure\CLI2` para evitar desbordamientos de memoria en CLI y garantizar compatibilidad con extensiones.
- **Python 3.10+ y Extensiones CLI:** Instalación de extensiones oficiales `containerapp` y `log-analytics` para habilitar el plano de control serverless.
- **Control de Acceso RBAC:** Permisos de `Contributor` o `Owner` en el Resource Group, con capacidad de asignación de roles (`AcrPull`) en el registro de destino.
- **Universal Execution Context:** Desacoplamiento total de credenciales efímeras de VM de laboratorio (`Admin / Pa55w.rd`) para operar en suscripciones Pay-As-You-Go, Visual Studio o Azure for Students.

AVISOS CRÍTICOS Y CONSIDERACIONES DE AZURE

- **Tolerancia a Latencia Transitoria:** La creación del Container Apps Environment puede reportar desconexiones transitorias en la terminal mientras el aprovisionamiento continúa en background.
- **Validación de Estado Succeeded:** Nunca desplegar la Container App hasta comprobar que `properties.provisioningState` del entorno devuelve `Succeeded` (evita error `ManagedEnvironmentNotProvisioned`).
- **Sintaxis de Shell:** Uso de acento grave (`` ` ``) para continuación de línea en PowerShell frente a barra invertida (`\`) en Bash / Zsh.
- **Proveedores de Recursos:** Registro mandatario de los namespaces `Microsoft.App`, `Microsoft.OperationalInsights` y `Microsoft.ContainerRegistry`.

Didáctica de Comandos Azure CLI: Jerarquía y Parámetros del Módulo

Comando / Jerarquía CLI	Parámetros Clave y Sintaxis	Propósito Arquitectónico y Comportamiento
`az group create / exists`	`--name <rg> --location <loc>`	Valida idempotencia y delimita el ciclo de vida y facturación de la solución.
`az acr create`	`--name <acr> --sku Basic --admin-enabled false`	Crea el registro privado seguro OCI forzando autenticación RBAC sin claves maestras.
`az acr build`	`--registry <acr> --image <name:v1> api/`	Compila la imagen en la nube mediante ACR Tasks sin requerir un Docker daemon local.
`az containerapp env create`	`--name <env> --resource-group <rg> --location <loc>`	Aprovisiona el límite de red compartido, dominio DNS y Log Analytics Workspace.
`az containerapp env show`	`--query properties.provisioningState -o tsv`	Inspecciona el estado asíncrono para verificar que el entorno alcanzó el estado Succeeded.
`az containerapp create`	`--ingress external --target-port 8000 --registry-identity system`	Despliega la app serverless, activa Managed Identity y asigna rol AcrPull en el ACR.
`az containerapp secret set`	`--secrets <secret-name>=<secret-value>`	Registra claves sensibles en el almacén cifrado del contenedor sin exponerlas en texto plano.
`az containerapp update`	`--set-env-vars KEY=secretref:<secret-name>`	Enlaza el secreto a una variable de entorno en memoria creando una nueva revisión inmutable.
`az containerapp revision list`	`--name <app> -g <rg> -o table`	Audita las revisiones activas con sufijos numéricos (`--0000002`) y división de tráfico.
`az containerapp logs show`	`--name <app> -g <rg> --follow`	Transmite en vivo la consola (stdout/stderr) para diagnosticar workers y peticiones HTTP.

Fase 1 (Task 0 & Pre-flight): Instalación CLI x64, Autenticación y Proveedores

Azure CLI • Prerrequisitos

Verifique la arquitectura oficial de 64 bits de Azure CLI, inicie sesión interactiva en Azure, instale las extensiones oficiales requeridas y registre los proveedores de recursos en la suscripción:

```
# 1. Confirmar compilación 64-bit de Azure CLI (instalada en C:\Program Files\Microsoft SDKs\Azure\CLI2)
az version

# 2. Iniciar sesión interactiva y seleccionar la suscripción de trabajo
az login

# 3. Instalar extensiones oficiales de Container Apps y Log Analytics
az extension add --name containerapp
az extension add --name log-analytics

# 4. Registrar los proveedores de recursos esenciales para el laboratorio
az provider register --namespace Microsoft.App
az provider register --namespace Microsoft.OperationalInsights
az provider register --namespace Microsoft.ContainerRegistry
```

Validación de Registro: El registro de proveedores puede demorar 1-2 minutos. Compruebe su estado con `az provider show -n Microsoft.App --query registrationState -o tsv` hasta obtener `Registered`.

Desglose Técnico del Código del Laboratorio

Análisis Arquitectónico de azdeploy.py, Dockerfile y Microservicio Flask (Task 1)

Análisis Arquitectónico de los Archivos del Proyecto

api/Dockerfile — CONTAINER IMAGE

Especifica la construcción de la imagen OCI del microservicio backend en Python:

- **Base Image:** `python:3.12-slim` para minimizar el vector de ataque y reducir el footprint de la imagen a ~145 MB.
- **Higiene de Capas:** `pip install --no-cache-dir` suprime cachés residuales de pip en la imagen final.
- **Puerto Estandarizado:** Declara `ENV PORT=8000` y `EXPOSE 8000` como contrato documental.
- **Servidor de Producción:** `gunicorn` con 2 workers asíncronos y `--access-logfile -` canalizando stdout hacia Azure Monitor.

api/main.py — FLASK BACKEND

Implementa los endpoints REST de inferencia y verificación para ACA:

- **Liveness Probe (`/health`):** Retorna `{"status": "healthy"}` consumido por los health probes de Container Apps.
- **Root Service Info (`/`):** Valida el modelo inyectado y verifica si el secreto está activo con `_embeddings_key_configured()` .
- **Document Ingestion (`/process`):** Endpoint POST que recibe texto o multipart y retorna metadatos de análisis documental simulado.
- **Almacenamiento Efímero:** Usa `/tmp/processed` adaptado al sistema de archivos efímero sin estado de ACA.

azdeploy.py — CLI ORCHESTRATION

Automatiza el aprovisionamiento de infraestructura base y variables:

- **Unicidad Global SHA-1:** `_derived_names()` genera sufijos hash de 8 caracteres basados en el Object ID de Entra ID (`acr{hash}` , `aca-env-{hash}`).
- **Plano Serverless:** Orquesta la creación de ACR SKU Basic y compila con `az acr build` mediante ACR Tasks.
- **Entorno de Contenedores:** Crea el Container Apps Environment y aprovisiona el Log Analytics Workspace.
- **Exportación Multiplataforma:** `write_env_files()` genera simultáneamente `.env` (Bash) y `.env.ps1` (PowerShell).

Preparación del Archivo de Despliegue en VS Code (Task 1)

Configuración Obligatoria

Descargue el paquete de código inicial, abra la carpeta del proyecto en VS Code y aplique las cuatro modificaciones normativas exigidas por la guía oficial de Microsoft Learn antes de iniciar la implementación:

Paso / Requisito	Código Original / Acción	Modificación Requerida y Rationale Arquitectónico
1. Parámetros Base `azdeploy.py` (Líneas 5-6)	`rg = "<your-resource-group-name>"` `location = "<your-azure-region>"`	Asigne su nombre de Resource Group y fije la región en `eastus`. Garantiza afinidad geográfica y cuotas de cómputo disponibles.
2. Find & Replace (Ctrl + H en VS Code)	Buscar: `gpt-5.4-mini`	Reemplazar por: `gpt-5.1` en todos los archivos. Alinea la solución con las versiones de despliegue admitidas en el ejercicio.
3. Límite de Tokens TPM `azdeploy.py` (Tras línea 22)	Línea 22: `MODEL_NAME = "gpt-5.1"`	Añada `TPM_LIMIT = "5000"` inmediatamente después de la línea 22 para controlar la cuota de inferencia del backend.
4. Inyección en Diccionario `azdeploy.py` (Líneas 209-220)	`write_env_files({ ... })` dentro de `create_containerapps_environment()`	Agregue `"TPM_LIMIT": TPM_LIMIT,` dentro del diccionario para exportar la variable a `.env` y `.env.ps1` .

```
# Descarga del paquete starter e inicio de despliegue en terminal de VS Code
# URL Oficial: https://github.com/MicrosoftLearning/mslearn-azure-ai/raw/main/downloads/python/aca-deploy-python.zip
python azdeploy.py

# Menú interactivo de azdeploy.py:
# [1] Create Azure Container Registry and build container image
# [2] Create Container Apps environment
# [3] Check deployment status
# [4] Exit
```

Regla de Aislamiento de Código: No modifique ninguna otra línea de `azdeploy.py` . La lógica de hashing SHA-1 y la resolución dinámica de ejecutables mediante `_resolve_exe()` son requeridas para la ejecución determinista en Windows, Linux y macOS.

Aprovisionamiento de Container Apps Environment: Azure Portal (GUI) vs. Azure CLI

Creación del Entorno Compartido de Ejecución: Límite de Red, Dominio DNS y Destino de Logs

1. Asistente Gráfico de Container Apps Environment en Azure Portal (GUI)

Azure Portal > Container Apps Environments > + Create

[Basics] [Networking] [Monitoring] [Tags] [Review + create]

Pestaña Wizard	Campo / Parámetro	Valor Recomendado	Rationale Arquitectónico
Basics	Resource Group	`<your-resource-group>`	Agrupación lógica y contención de costes para la solución serverless.
Basics	Environment Name	`aca-env-<hash>`	Nombre global único que define el subdominio DNS de las aplicaciones alojadas.
Basics	Region	`East US`	Alineación de latencia y disponibilidad de cuotas de cómputo para Container Apps.
Basics	Plan Type / Workload	`Consumption (Serverless)`	Escalado dinámico a cero sin costes fijos de nodos subyacentes.
Networking	Virtual Network (VNet)	`No (Default Managed VNet)`	Azure gestiona el aislamiento de red perimetral y el proxy Envoy frontal.
Monitoring	Log Destination	`Log Analytics Workspace`	Destino unificado para persistencia de logs de consola y eventos de réplica KEDA.
Review + create	Validation Summary	`Validation Passed`	Comprobación de cuotas de suscripción y directivas de gobernanza de Azure Policy.

2. Automatización y Equivalencia en Azure CLI

Comando de Creación y Verificación de Estado en Azure CLI

az containerapp env

Cree el entorno de aplicaciones de contenedor mediante CLI e inspeccione el estado de aprovisionamiento asíncrono para prevenir fallos en la fase de despliegue de la app:

```
# 1. Crear el Container Apps Environment en la región especificada
az containerapp env create `
    --name $env:ACA_ENVIRONMENT `
    --resource-group $env:RESOURCE_GROUP `
    --location eastus

# 2. Consultar el estado de aprovisionamiento del entorno (esperar hasta que devuelva "Succeeded")
az containerapp env show `
    --name $env:ACA_ENVIRONMENT `
    --resource-group $env:RESOURCE_GROUP `
    --query properties.provisioningState -o tsv
```

⚠ **Fallo Crítico Evitable (ManagedEnvironmentNotProvisioned):** Si la opción 2 del script reporta error de conexión por latencia de Azure, el entorno suele seguir creándose en background. Si ejecuta `az containerapp create` mientras el entorno está en estado `Waiting`, el comando fallará de inmediato. Ejecute el comando `az containerapp env show` arriba indicado hasta confirmar `Succeeded` antes de avanzar.

3. Inspección Post-Aprovisionamiento en Azure Portal (GUI)

HOJA OVERVIEW

Muestra el **Default Domain** del entorno (`*.eastus.azurecontainerapps.io`), la IP pública del balanceador de carga de entrada y el estado operativo `Succeeded`.

LOG STREAM & CONSOLE

Permite la inspección en tiempo real de eventos del plano de control, orquestación de réplicas, pull de imágenes y registros del sistema de la infraestructura subyacente.

LOG ANALYTICS WORKSPACE

Recurso aprovisionado automáticamente para almacenar tablas analíticas (`ContainerAppConsoleLogs_CL` y `ContainerAppSystemLogs_CL`) accesibles vía KQL.

Aprovisionamiento de Servicios Base con azdeploy.py y Gestión de Entorno

Ejecución de Pasos 1 y 2 del Script Automatizado y Carga de Variables (Task 1)

Paso 2.1: Crear Azure Container Registry y Compilar la Imagen (Opción 1)

Automatización CLI en azdeploy.py (Opción 1)

ACR Tasks

El script valida el Resource Group, aprovisiona un ACR con clave de administrador deshabilitada y ejecuta la compilación en la nube:

```
# Comandos ejecutados internamente por la opción 1:
az acr create `
    --resource-group <rg-name> `
    --name <acr-name> `
    --sku Basic `
    --admin-enabled false

az acr build `
    --resource-group <rg-name> `
    --registry <acr-name> `
    --image ai-api:v1 `
    --file api/Dockerfile `
    --no-logs api/
```

Serverless Build: `az acr build` empaqueta la carpeta `api/` en un tarball y lo envía al motor de compilación de ACR Tasks, eliminando la necesidad de Docker Desktop en el equipo cliente.

VERIFICACIÓN EN AZURE PORTAL (GUI)

- **Navegación:** `Azure Portal > Container registries > <acr-name>`.
- **Repositorios y Etiquetas:** En la hoja *Repositories*, confirme la presencia de `ai-api` con la etiqueta `v1` y tamaño aproximado de 145 MB.
- **Claves de Acceso Deshabilitadas:** En *Access keys*, compruebe que *Admin user* está desactivado. Esto cumple el estándar Zero-Trust evaluado en el examen AI-200.
- **Historial de Tareas:** En *Tasks > Runs*, examine el ID de ejecución de ACR Tasks, verificando estado `Successful` y tiempo de compilación.

Paso 2.2: Crear el Entorno y Cargar Variables de Sesión (Opciones 2 y 4)

Ejecución del Script y Carga de Variables en Terminal

.env.ps1 / .env

Ejecute la opción 2 para crear el Container Apps Environment. Al finalizar, seleccione la opción 4 para salir del script y cargue las variables generadas en su sesión:

```
# 1. En la consola interactiva de azdeploy.py:
# Introduzca '2' para lanzar "Create Container Apps environment"
# Al terminar, introduzca '4' para salir del script ("Exit")

# 2. Cargar las variables de entorno en la sesión actual de PowerShell:
. .\.env.ps1

# Nota equivalente para entornos Linux / macOS (Bash / Zsh):
# source .env

# 3. Validar que las variables esenciales se han exportado correctamente
Write-Output "ACR: $env:ACR_NAME | Environment: $env:ACA_ENVIRONMENT | App: $env:CONTAINER_APP_NAME"
```

Tabla de Mapeo de Variables de Entorno Exportadas

Variable de Entorno	Valor Típico / Formato	Rol en la Arquitectura de Contenedores
`$env:RESOURCE_GROUP`	`rg-ai200-lab`	Nombre del grupo de recursos que alberga todos los componentes del laboratorio.
`$env:ACR_NAME`	`acr1a2b3c4d`	Nombre global del Azure Container Registry (alfanumérico único por SHA-1).
`$env:ACR_SERVER`	`acr1a2b3c4d.azurecr.io`	FQDN del servidor de login del registro OCI para operaciones de docker pull.
`$env:ACA_ENVIRONMENT`	`aca-env-1a2b3c4d`	Nombre del Container Apps Environment que provee el límite de red y DNS.
`$env:CONTAINER_APP_NAME`	`ai-api`	Identificador del microservicio dentro del entorno de Container Apps.
`$env:CONTAINER_IMAGE`	`ai-api:v1`	Nombre de la imagen y etiqueta inmutable construida por ACR Tasks.
`$env:TARGET_PORT`	`8000`	Puerto interno de escucha en el contenedor mapeado al balanceador Envoy.
`$env:MODEL_NAME`	`gpt-5.1`	Nombre del modelo de inferencia inyectado como variable de configuración.
`$env:EMBEDDINGS_API_KEY`	`demo-key-12345`	Valor sensible registrado como secreto en Container Apps y mapeado por `secretref`.
`$env:TPM_LIMIT`	`5000`	Límite de Tokens por Minuto inyectado tras la modificación de la Task 1.

Creación de Container App con Ingress y Autenticación Gestionada

Despliegue del Microservicio con Identidad del Sistema y Rol AcrPull Automático (Task 2)

1. Despliegue Automatizado con Azure CLI: Comando Maestro y Anatomía

`az containerapp create (Task 2)`

System Managed Identity • AcrPull

Cree la aplicación de contenedor configurando el balanceador Envoy público (ingress externo), el puerto de escucha y la identidad administrada para la extracción segura de la imagen desde ACR:

```
az containerapp create `
    --name $env:CONTAINER_APP_NAME `
    --resource-group $env:RESOURCE_GROUP `
    --environment $env:ACA_ENVIRONMENT `
    --image "$env:ACR_SERVER/$env:CONTAINER_IMAGE" `
    --ingress external `
    --target-port $env:TARGET_PORT `
    --env-vars MODEL_NAME=$env:MODEL_NAME `
    --registry-server "$env:ACR_SERVER" `
    --registry-identity system
```

Parámetro / Flag	Valor en Laboratorio	Comportamiento Arquitectónico
`--registry-identity system`	`system`	Habilita la identidad administrada asignada por el sistema y asigna automáticamente el rol `AcrPull` en el ACR.
`--ingress external`	`external`	Expone la aplicación a Internet mediante el proxy inverso Envoy administrado con terminación TLS 1.3 automática.
`--target-port`	`8000`	Enruta las solicitudes HTTP entrantes al socket TCP interno de Gunicorn configurado en el contenedor.
`--env-vars`	`MODEL_NAME=gpt-5.1`	Inyecta la variable no confidencial en el contenedor para la primera revisión de la aplicación.

2. Creación Paso a Paso en Azure Portal (GUI): Asistente de Container App

Azure Portal > Container Apps > + Create

[Basics] [Container] [Ingress] [Review + create]

Pestaña Wizard	Campo / Parámetro	Valor Seleccionado	Rationale de Configuración
Basics	Container App Name	`ai-api`	Identificador del recurso que compone el FQDN de la aplicación.
Basics	Container Apps Environment	`aca-env-<hash>`	Selecciona el entorno previamente validado en estado Succeeded.
Container	Image Source	`Azure Container Registry`	Conecta de forma nativa con el registro privado OCI en la suscripción.
Container	Registry Identity	`System-assigned managed identity`	Elimina credenciales maestras y delega la autenticación a Entra ID.
Container	Image & Tag	`ai-api:v1`	Imagen de producción construida previamente mediante ACR Tasks.
Ingress	Ingress Traffic	`Accepting traffic from anywhere`	Habilita el balanceador Envoy público externo en puerto 443.
Ingress	Target Port	`8000`	Mapeo exacto al puerto expuesto por Gunicorn en el Dockerfile.

3. Diagnóstico de Errores Comunes de Red y Registro

⚠ **Fallo de Conexión en Creación (NameResolutionError / Max Retries Exceeded):** Si el comando `az containerapp create` reporta un error de red o timeout mientras espera la respuesta de Azure, la Container App puede haberse creado en el plano de control pero con una configuración incompleta (imagen de marcador de posición y sin credenciales de registro). Compruebe el estado ejecutando:
`az containerapp show -n $env:CONTAINER_APP_NAME -g $env:RESOURCE_GROUP --query "{image:properties.template.containers[0].image, registries:properties.configuration.registries}"`
Si `registries` devuelve `null`, configure el registro manualmente con `az containerapp registry set --identity system` y actualice la imagen con `az containerapp update --image ...` en lugar de asumir que la creación falló totalmente.

Gestión de Secretos, Variables de Entorno y Modelo de Revisiones

Configuración Segura de Claves de API, Desacoplamiento secretref y Ciclo de Vida (Task 2)

1. Configuración de Secretos y Enlace Runtime (Task 2)

Configuración de Secreto e Inyección como Variable de Entorno

Secret Reference (secretref)

Almacene la clave de API como secreto seguro en Azure Container Apps y vincúlela a una variable de entorno mediante la sintaxis `secretref:`. Esta operación reinicia el microservicio creando una nueva revisión inmutable:

```
# 1. Almacenar el secreto en el almacén seguro de Azure Container Apps
az containerapp secret set `
    -n $env:CONTAINER_APP_NAME `
    -g $env:RESOURCE_GROUP `
    --secrets embeddings-api-key=$env:EMBEDDINGS_API_KEY

# 2. Vincular el secreto a la variable de entorno EMBEDDINGS_API_KEY (genera una nueva revisión)
az containerapp update `
    -n $env:CONTAINER_APP_NAME `
    -g $env:RESOURCE_GROUP `
    --set-env-vars EMBEDDINGS_API_KEY=secretref:embeddings-api-key
```

Arquitectura de Seguridad Zero-Trust: La directiva `secretref:` asegura que el valor del secreto nunca quede expuesto en el portal en texto plano, en la plantilla ARM/Bicep ni en el historial de comandos del shell. El runtime de Container Apps inyecta el valor directamente en el espacio de memoria del proceso en tiempo de ejecución.

2. Modelo de Revisiones Inmutables en Azure Container Apps

Auditoría de Revisiones

az containerapp revision

Confirme la creación de la nueva revisión tras la inyección del secreto:

```
az containerapp revision list `
    -n $env:CONTAINER_APP_NAME `
    -g $env:RESOURCE_GROUP `
    -o table
```

Sufijo de Revisión: El nombre de la revisión termina con un sufijo incremental como `--0000002`, indicando que es la segunda instantánea inmutable del contenedor.

MODOS DE REVISIÓN Y DIVISIÓN DE TRÁFICO

- **Single Revision Mode (Por Defecto):** Toda actualización redirige el 100% del tráfico a la nueva revisión una vez que los health probes responden exitosamente (Zero-Downtime Deployment).
- **Multiple Revision Mode:** Permite tener múltiples revisiones activas concurrentemente para estrategias avanzadas de despliegue:
  - *Blue/Green:* Despliegue de versión sin tráfico, validación de endpoints y cambio instantáneo al 100%.
  - *Canary Splitting:* División porcentual de tráfico (ej. 80% a v1, 20% a v2) con `az containerapp ingress traffic set`.

3. Configuración y Visualización en Azure Portal (GUI)

HOJA SECRETS

Ubicada en *Application > Secrets*. Muestra el nombre del secreto `embeddings-api-key`, su tipo (Container App Secret o Key Vault Reference) y oculta el valor sensible con asteriscos.

HOJA CONTAINERS

En *Application > Containers > Edit and deploy*, inspeccione las variables de entorno. `EMBEDDINGS_API_KEY` figura con origen `Secret reference` vinculado a `embeddings-api-key`.

REVISION MANAGEMENT

En *Application > Revision management*, visualice el árbol de revisiones. Muestra la revisión `--0000001` como inactiva (0% de tráfico) y `--0000002` activa con el 100% del tráfico asignado.

Verificación Operativa, Pruebas de API y Observabilidad en Tiempo Real

Validación de Endpoints, Ingesta de Documentos y Análisis de Logs de Consola (Task 3)

1. Obtención de FQDN y Validación de Endpoints REST (Task 3)

Pruebas Funcionales con PowerShell (Invoke-RestMethod)

Health & Ingestion Testing

Recupere el nombre de dominio completo (FQDN) público asignado a la Container App y valide el sondeo de salud, la configuración de secretos y el procesamiento de documentos:

```powershell
# 1. Recuperar el FQDN de la Container App y almacenarlo en variable
$FQDN = az containerapp show `
    -n $env:CONTAINER_APP_NAME `
    -g $env:RESOURCE_GROUP `
    --query properties.configuration.ingress.fqdn -o tsv

Write-Output $FQDN

# 2. Validar el sondeo de vitalidad (debe retornar {"status": "healthy"})
Invoke-RestMethod -Uri "https://$FQDN/health"

# 3. Validar el estado del secreto llamando al endpoint raíz (debe confirmar embeddings_api_key_configured: true)
Invoke-RestMethod -Uri "https://$FQDN/"

# 4. Crear un archivo de prueba para simular la ingesta de documentos
Set-Content -Path document.txt -Value "This is a test document for the Azure Container Apps exercise."

# 5. Probar el endpoint de inferencia y procesamiento documental
Invoke-RestMethod -Uri "https://$FQDN/process" `
    -Method Post `
    -ContentType "text/plain" `
    -Body (Get-Content -Raw document.txt)
```

2. Observabilidad, Streaming de Logs y Diagnóstico en Tiempo Real

Streaming de Logs en Vivo

az containerapp logs

Transmita en vivo stdout y stderr desde los contenedores activos:

```powershell
az containerapp logs show `
    -n $env:CONTAINER_APP_NAME `
    -g $env:RESOURCE_GROUP `
    --follow
```

Señales Operativas Clave: Busque mensajes de inicio de Gunicorn como `Listening at: http://0.0.0.0:8000`, workers iniciados y líneas de registro HTTP (`GET /health 200`, `POST /process 200`).

PERSISTENCIA Y RÉPLICAS KEDA

- **Log Analytics Workspace:** Para análisis histórico y auditoría forense, los registros se almacenan permanentemente en las tablas:
  - `ContainerAppConsoleLogs_CL` (salida stdout/stderr de la app).
  - `ContainerAppSystemLogs_CL` (eventos del orquestador ACA, escalado y pulls de imagen).
- **Inspección de Réplicas KEDA:**

```powershell
az containerapp replica list -n $env:CONTAINER_APP_NAME -g $env:RESOURCE_GROUP -o table
```

Permite confirmar el recuento de pods activos, arranques en frío (cold starts) o escalado a cero (scale-to-zero) en ausencia de peticiones HTTP.

Diagrama de Arquitectura Lógica de la Solución (Azure Container Apps)

Topología de Componentes, Flujo de Datos y Desacoplamiento de Servicios Serverless

CASO DE USO REAL: INGESTA Y PROCESAMIENTO INTELIGENTE DE DOCUMENTOS CON MODELOS DE LENGUAJE

Problemática Empresarial: Una organización necesita un backend elástico para procesar documentos no estructurados y generar embeddings vectoriales. Los servidores tradicionales incurren en altos costes por infraestructura ociosa fuera del horario comercial y representan un riesgo de seguridad al usar contraseñas maestras en código para acceder al registro de imágenes.
Solución de Arquitectura en Azure: Despliegue de un microservicio Python/Flask en **Azure Container Apps** bajo el perfil Serverless Consumption. La imagen se almacena en **Azure Container Registry** y se extrae mediante **System-Assigned Managed Identity** con el rol `AcrPull`. Las credenciales de IA se aíslan mediante `secretref:` en variables de entorno, y el backend escala automáticamente de 0 a N réplicas con KEDA según el tráfico HTTP entrante.

BLUEPRINT DE ARQUITECTURA LÓGICA (DRAW.IO CANVAS ESTILO VECTORIAL)

Zero-Trust & Serverless Microservices

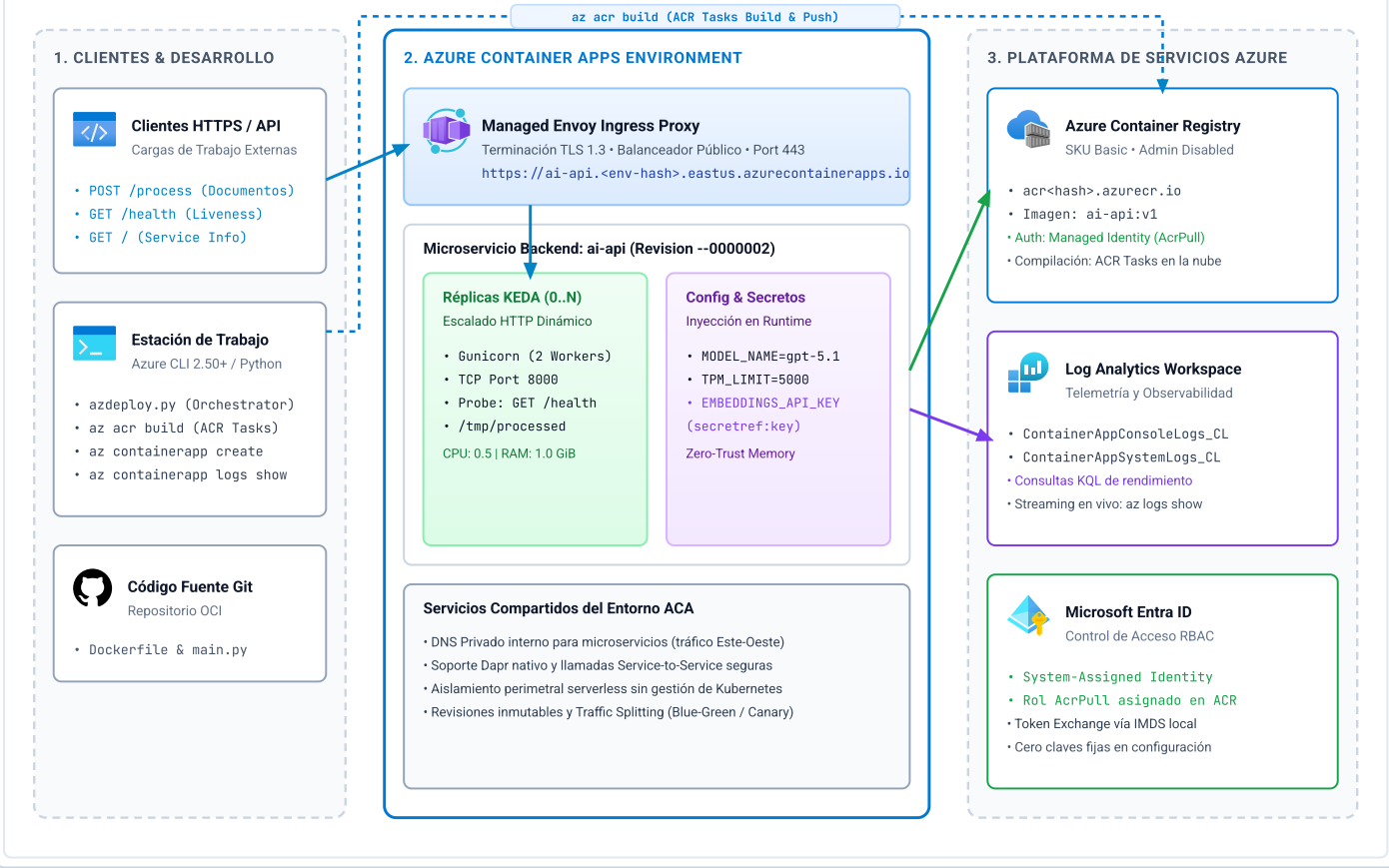


Diagrama de Arquitectura Física y Red de Azure Container Apps

Aislamiento de Red, Terminación TLS, Tráfico Norte-Sur / Este-Oeste y Topología VNet

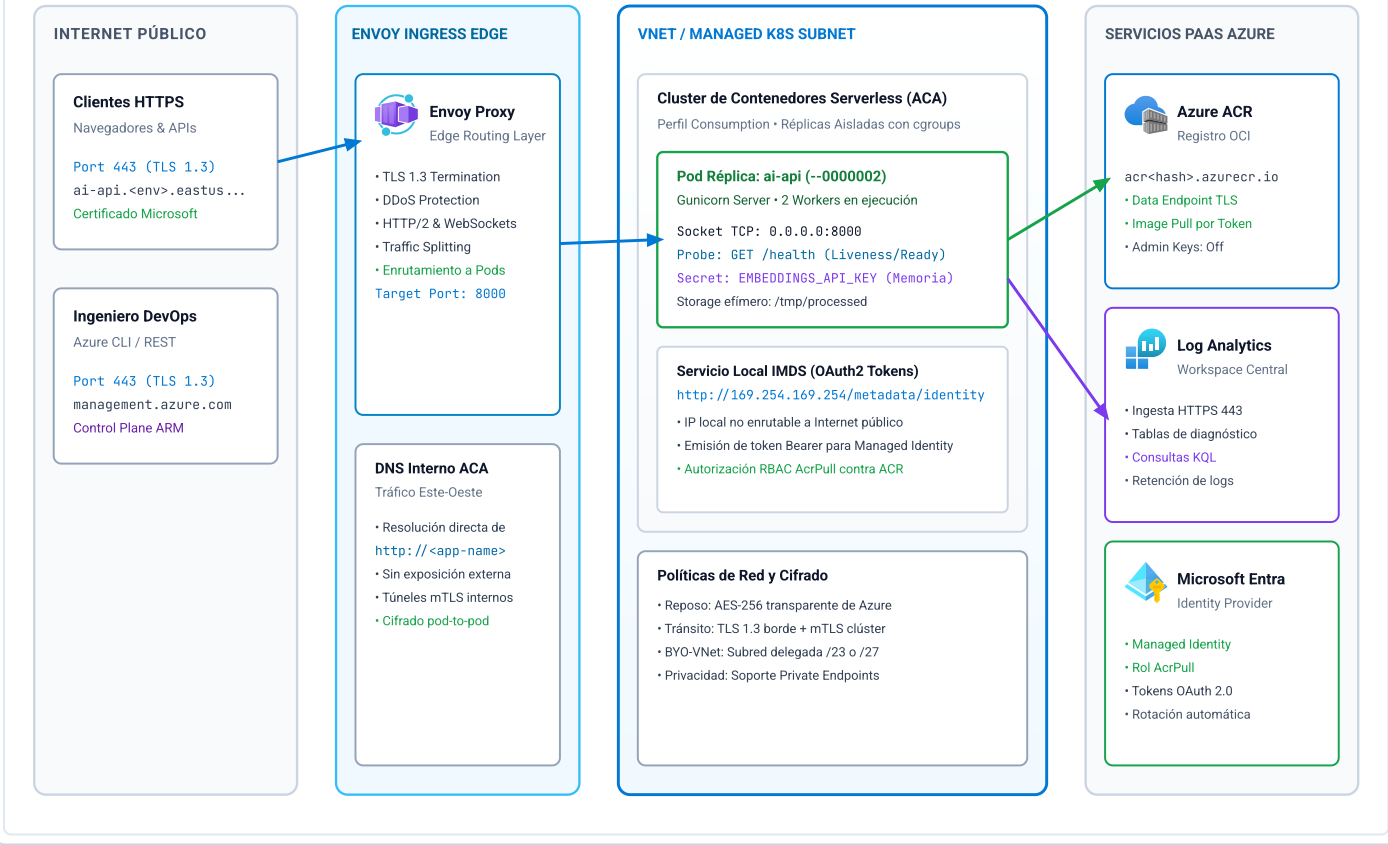
TOPOLOGÍA DE RED Y FRONTERAS DE AISLAMIENTO FÍSICO

Aislamiento de Red y Subred Delegada: Azure Container Apps opera dentro de un entorno Kubernetes serverless completamente administrado por Microsoft. El Container Apps Environment puede inyectarse en una subred dedicada de una VNet del cliente (`/23` para perfiles Consumption o `/27` para Workload Profiles dedicados) para aislar completamente las comunicaciones privadas.

Tráfico Norte-Sur vs Este-Oeste: El tráfico entrante de Internet (Norte-Sur) termina en el controlador de entrada Envoy con TLS 1.3 gestionado y certificados automáticos. La comunicación entre microservicios dentro del mismo entorno (Este-Oeste) se realiza a través de DNS privado interno y túneles mTLS automáticos, impidiendo la interceptación de paquetes entre réplicas.

TOPOLOGÍA DE RED Y PLANO FÍSICO (DRAW.IO CANVAS ESTILO VECTORIAL)

TLS 1.3 • Envoy Ingress • IMDS 169.254.169.254 • mTLS



Limpieza de Recursos y Diagnóstico Oficial (Troubleshooting)

Procedimiento de Desaprovisionamiento y Resolución de Incidentes Operativos en ACA

Fase de Limpieza de Recursos (Clean up resources)

Eliminación del Grupo de Recursos en Segundo Plano

Desaprovisionamiento

Al finalizar las pruebas del laboratorio, elimine todos los recursos creados para evitar consumos no deseados de cómputo y cuotas en la suscripción:

```
# Eliminar el grupo de recursos y todos los servicios en cascada en segundo plano
az group delete --name <rg-name> --no-wait --yes
```

⚠ **Precaución de Gobernanza:** La eliminación del Resource Group destruye de forma irreversible el ACR, las imágenes OCI, el Container Apps Environment, la Container App y el Log Analytics Workspace.

Guía Oficial de Resolución de Problemas (Troubleshooting de Microsoft Learn)

Síntoma / Error Observado	Causa Raíz Arquitectónica	Procedimiento de Diagnóstico y Mitigación
Fallo o Tarea Encolada en ACR Build (Estado `Queued` prolongado)	Agentes de compilación de ACR Tasks saturados o contexto de directorio incorrecto.	Liste ejecuciones con `az acr task list-runs --registry $ACR_NAME -o table`. Cancele la tarea colgada con `az acr task cancel-run` y reintente el build directamente.
ManagedEnvironmentNotProvisioned (Al crear la Container App)	El Container Apps Environment aún no ha alcanzado el estado de aprovisionamiento `Succeeded`.	Ejecute `az containerapp env show -n $ACA_ENV -g $RG --query properties.provisioningState -o tsv` y espere a que retorne `Succeeded` antes de crear la app.
ImagePullBackOff / 403 Forbidden (Error al extraer imagen desde ACR)	Falta el rol RBAC `AcrPull` o la identidad administrada no se ha propagado en Entra ID.	Compruebe la identidad con `az containerapp identity show`. Valide que la asignación de rol en el ACR tenga el scope adecuado y permita 1-2 minutos de propagación.
HTTP 502 / 503 Bad Gateway (Tras el despliegue inicial)	El contenedor está realizando un arranque en frío (cold start) o el puerto de Gunicorn no coincide con `--target-port`.	Inspeccione los logs en tiempo real con `az containerapp logs show --follow`. Confirme que Gunicorn está escuchando en `0.0.0.0:8000` y verifique revisiones activas.
Creación Incompleta de App (NameResolutionError / Max retries)	Latencia de Azure que provocó una creación parcial sin credenciales de registro asociadas.	Consulte `az containerapp show --query properties.configuration.registries`. Si devuelve `null`, configure el registro con `az containerapp registry set`.
Secreto No Detectado en / (`embeddings_api_key_configured: false`)	El secreto fue creado en ACA pero no se vinculó a la variable mediante la sintaxis `secretref:`.	Verifique con `az containerapp secret list -o table` y reejecute `az containerapp update --set-env-vars EMBEDDINGS_API_KEY=secretref:embeddings-api-key`.

REGLAS DE ORO DE ARQUITECTURA PARA EL EXAMEN AI-200 (PPT Y MICROSOFT LEARN)

- **1. Identidad Administrada en ACR:** Para servicios de IA de producción que extraen múltiples imágenes, utilice siempre System-Assigned Managed Identity con `AcrPull` en lugar de credenciales maestras de administrador.
- **2. Inmutabilidad de Revisiones:** Cada modificación de imagen, secreto o variable de entorno crea una nueva revisión inmutable en Container Apps. Esto permite auditoría de versiones y reversión segura (rollback).
- **3. Desacoplamiento de Secretos:** Nunca inserte API keys en la imagen de contenedor ni en el archivo YAML en texto plano. Utilice el almacén de secretos de Container Apps y haga referencia a ellos con `secretref:`.
- **4. Delimitación de Entorno:** Agrupe aplicaciones que comparten ciclo de vida, dominio de red o destino de observabilidad en el mismo Container Apps Environment para habilitar comunicación Este-Oeste por DNS interno.

Preguntas Oficiales de Examen de Certificación (AI-200 Skills Matrix)

Knowledge Check Oficial de Microsoft Learn, Diapositivas PPTX y Matriz de Ponderación

DISTRIBUCIÓN DE HABILIDADES EVALUADAS EN EL EXAMEN OFICIAL AI-200 (GUÍA DE ESTUDIO MICROSOFT LEARN)

AI-200 Study Guide

1. Contenedores Azure — 20% – 25%	2. Datos de IA en Azure — 25% – 30%	3. Conexión de Servicios — 20% – 25%	4. Seguridad y Monitoreo — 20% – 25%
★ **Evaluado en Lab 3:** Container Apps, Entornos, Revisiones, Ingress, Managed Identity y Secretos.	Cosmos DB NoSQL, PostgreSQL con `pgvector`, caché semántica y similitud en Managed Redis.	Mensajería desacoplada con Service Bus, Event Grid y arquitecturas dirigidas por eventos en Functions.	Identidades administradas Entra ID, Key Vault, App Configuration, OpenTelemetry y consultas KQL.

Pregunta 1: Despliegue Declarativo (Slide 12 Oficial)

YAML & Code Review

Quieres una implementación coherente y controlada por código fuente para una Container App, de modo que la configuración se revise como código (GitOps / PR review). ¿Qué enfoque admite mejor ese objetivo?

A) Usar `az containerapp create --yaml` con un archivo YAML en el control de código fuente.

B) Usar `az containerapp update --set-env-vars` para todos los cambios.

C) Basarse en etiquetas de imagen y volver a implementar con `az containerapp update --image`.

✓ **Respuesta Correcta: A) Usar az containerapp create --yaml con un archivo YAML**

Justificación: El uso de `--yaml` permite almacenar la definición completa de la aplicación (contenedores, recursos, ingress, secretos y reglas de escala) en Git, facilitando revisiones por pares, reproducibilidad y auditoría CI/CD.

Pregunta 2: Diagnóstico Rápido de Errores (Slide 12 Oficial)

Logs & Troubleshooting

Una aplicación contenedora no se puede iniciar después de actualizar la imagen. Necesitas comentarios rápidos para diagnosticar el problema. ¿Qué comando es el mejor primer paso?

A) Ejecutar `az containerapp logs show` para transmitir la salida de consola de la aplicación.

B) Eliminar y volver a crear el Container Apps Environment completo.

C) Escalar el número de réplicas a cero y luego volver a escalar a diez.

✓ **Respuesta Correcta: A) az containerapp logs show**

Justificación: `az containerapp logs show` es la señal más rápida para exponer excepciones en tiempo de inicio, variables de entorno faltantes o errores de conexión en el socket interno de la aplicación.

Pregunta 3: Extracción Segura de Imágenes (Slide 9)

Managed Identity & AcrPull

Se requiere configurar la autenticación de extracción de imágenes desde un ACR privado para microservicios de IA de producción sin gestionar ni rotar credenciales. ¿Qué configuración es la recomendada?

A) Habilitar la cuenta de administrador en el ACR y suministrar usuario y contraseña.

B) Habilitar Managed Identity en la app y concederle el rol `AcrPull` sobre el ACR.

C) Almacenar las credenciales de un Service Principal en un archivo de texto en el contenedor.

✓ **Respuesta Correcta: B) Managed Identity con rol AcrPull**

Justificación: La identidad administrada asignada por el sistema elimina credenciales estáticas. Concede el rol de mínimo privilegio `AcrPull` y rota automáticamente los tokens de Entra ID.

Pregunta 4: Gestión de Secretos en Runtime (Slide 8)

Secret References (secretref)

Una API requiere una clave de API confidencial de un proveedor de IA. Se debe rotar la clave sin recompilar la imagen de contenedor ni exponerla en el historial de comandos. ¿Qué patrón cumple esto?

A) Escribir la clave de API directamente en la directiva ENV del Dockerfile.

B) Registrar el secreto en Container Apps y mapearlo a una env var con `secretref:`.

C) Pasar la clave como parámetro en la URL de consulta (query parameter) del endpoint.

✓ **Respuesta Correcta: B) Registrar secreto y mapear con secretref:**

Justificación: `secretref:` mantiene los valores confidenciales fuera de los archivos de manifiesto y del historial del shell, permitiendo actualizar el secreto sin tocar el código de la app.

Pregunta 5: Límite de Red de Entorno (Slide 6)

Environment Boundary

Tres microservicios deben comunicarse entre sí de forma privada sin exponer sus endpoints a Internet público. ¿Cómo deben desplegarse en Azure Container Apps?

A) En el mismo Container Apps Environment con Ingress interno activado.

B) En tres Container Apps Environments separados en diferentes regiones.

C) Usando direcciones IP públicas con autenticación básica HTTP.

✓ **Respuesta Correcta: A) Mismo Environment con Ingress interno**

Justificación: Un Container Apps Environment actúa como límite de red compartido con DNS interno, permitiendo resolución directa de nombres de servicio sin salir a la red pública.

INTERACTIVE KNOWLEDGE CHECK CON MICROSOFT COPILOT (SLIDE 13 OFICIAL)

Prompt Oficial de Examen para Copilot Chat:

```
"As an Azure Container Apps (ACA) expert, please create three multiple-choice questions that assess knowledge of deploying containers to ACA. Present one question at a time and wait for my response before continuing. After each answer, provide a brief explanation of the correct choice before moving on to the next question."
```

Instrucción pedagógica: Ejecute este prompt en Copilot para reforzar los conceptos antes del examen AI-200.

Glosario de Términos, Conceptos Clave y Siglas de Azure

Definiciones Normativas de Azure Container Apps, Diccionario de Siglas y Mapa Curricular Oficial

1. Conceptos Arquitectónicos Centrales de Azure Container Apps

CONTAINER APPS ENVIRONMENT
Límite lógico de red y aislamiento que agrupa aplicaciones de contenedor, proporcionando dominio DNS interno compartido, VNet y destino unificado de Log Analytics.

REVISION (REVISIÓN INMUTABLE)
Instantánea inmutable con versiones de una Container App generada automáticamente ante cambios de imagen o configuración, con soporte de división de tráfico.

MANAGED INGRESS (ENVOY)
Controlador de entrada serverless que proporciona terminación TLS 1.3 automática, certificados SSL gratuitos, balanceo de carga HTTP/2 y enrutamiento externo o interno.

KEDA (ESCALADO POR EVENTOS)
Motor de autoescalado basado en eventos integrado en ACA que ajusta las réplicas dinámicamente de 0 a N según peticiones HTTP concurrentes o mensajes en cola.

SECRET REFERENCE (SECRETREF)
Directiva declarativa que inyecta valores confidenciales en variables de entorno en memoria sin exponerlos en el portal, plantillas ARM ni historial del shell.

SYSTEM-ASSIGNED IDENTITY
Identidad de Microsoft Entra ID ligada al ciclo de vida del contenedor que permite autenticarse sin contraseñas contra Azure Container Registry con el rol AcrPull.

SCALE-TO-ZERO (ESCALADO A 0)
Capacidad serverless de apagar todas las réplicas en ausencia de tráfico entrante, suspendiendo totalmente los costes de procesamiento hasta la siguiente solicitud.

LOG ANALYTICS WORKSPACE
Repositorio analítico centralizado donde persisten los registros de consola (stdout/stderr) y del sistema de ACA, consultables mediante lenguaje KQL.

WORKLOAD PROFILES
Modelo de dimensionamiento de cómputo que permite seleccionar entre perfiles Serverless Consumption o perfiles dedicados con hardware optimizado para memoria o GPU.

2. Diccionario de Siglas y Acrónimos Técnicos del Módulo

Sigla	Significado y Rol en Azure
ACA	**Azure Container Apps:** Plataforma de contenedores serverless basada en Kubernetes administrado.
ACR	**Azure Container Registry:** Registro privado administrado para almacenamiento y compilación de imágenes OCI.
KEDA	**Kubernetes Event-driven Autoscaling:** Componente para escalado elástico basado en eventos y HTTP.
RBAC	**Role-Based Access Control:** Control de acceso basado en roles (`AcrPull`, `Contributor`).
MI / MSI	**Managed Identity:** Identidades sin contraseñas administradas directamente por Microsoft Entra ID.
FQDN	**Fully Qualified Domain Name:** Nombre de dominio completo (ej. `<app>.<env>.eastus.azurecontainerapps.io`).
DAPR	**Distributed Application Runtime:** Framework opcional de microservicios para pub/sub y estado.
KQL	**Kusto Query Language:** Lenguaje de consulta analítica para telemetría en Log Analytics.
OCI	**Open Container Initiative:** Especificaciones estándar para formatos de imagen y runtimes.
TPM	**Tokens Per Minute:** Métrica de cuota y límite de procesamiento en servicios de IA en Azure.

Sigla	Significado y Rol en Azure
TLS	**Transport Layer Security:** Cifrado criptográfico TLS 1.3 para canales HTTPS de entrada.
IMDS	**Instance Metadata Service:** Endpoint local `169.254.169.254` para emisión de tokens Entra ID.
VNET	**Virtual Network:** Red privada virtual de Azure para aislamiento de entornos ACA.
ARM	**Azure Resource Manager:** Plano de control declarativo para aprovisionamiento en Azure.
REST	**Representational State Transfer:** Protocolo estándar de endpoints de API sin estado sobre HTTP.
WSGI	**Web Server Gateway Interface:** Interfaz entre servidores web (Gunicorn) y aplicaciones Python.
SKU	**Stock Keeping Unit:** Nivel de tarificación y capacidades del servicio (Basic, Premium, Consumption).
TSV	**Tab-Separated Values:** Formato de salida tabular para extracción de valores en Azure CLI.
JSON	**JavaScript Object Notation:** Formato estándar de intercambio de datos en APIs REST y CLI.
CVE	**Common Vulnerabilities and Exposures:** Identificador público de vulnerabilidades de seguridad.

3. Mapa Curricular y Rutas de Aprendizaje Oficiales de Microsoft Learn (Curso AI-200T00-A)

Curso Oficial Microsoft: AI-200T00-A: Desarrollo de soluciones en la nube de IA en Azure (5 días / 120 horas lectivas)

Certificación Asociada: Microsoft Certified: Azure AI Cloud Developer Associate (Examen AI-200)

FASE 1: CÓMPUTO & CONTENEDORES (DOM. 1)
- **Módulo 1:** Hospedaje de contenedores en Azure (ACR, ACR Tasks serverless, OCI).
- **Módulo 2:** Implementación en Azure App Service (Linux PaaS, Managed Identity).
- **Módulo 3:** Implementación y escalado con KEDA en Azure Container Apps (ACA). **[★ Este Lab]**

FASE 2: DATOS & EMBEDDINGS (DOM. 2)
- **Módulo 4:** Soluciones de IA con Azure Cosmos DB for NoSQL y vector search.
- **Módulo 5:** Bases de datos vectoriales con Azure Database for PostgreSQL (`pgvector`).
- **Módulo 6:** Optimización de latencia y caché semántica con Azure Managed Redis.

FASE 3: INTEGRACIÓN & DEVOPS (DOM. 3 Y 4)
- **Módulo 7:** Integración desacoplada con Azure Service Bus, Event Grid y Functions.
- **Módulo 8:** Gestión de secretos y feature flags en Azure Key Vault y App Configuration.
- **Módulo 9:** Observabilidad de extremo a extremo, OpenTelemetry y diagnóstico KQL.

Microsoft Learn Study Guide: https://learn.microsoft.com/es-es/credentials/certifications/resources/study-guides/ai-200

Microsoft Worldwide Learning (WWL) • Curso AI-200T00-A